

INF 154

LECTURE 2

- What is programming?
- Input-Processing-Output
- Data: variables and constants
- String manipulation and simple arithmetic



UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

Denkleiers • Leading Minds • Dikgopolo tša Dihalefi

After this lecture you should ...

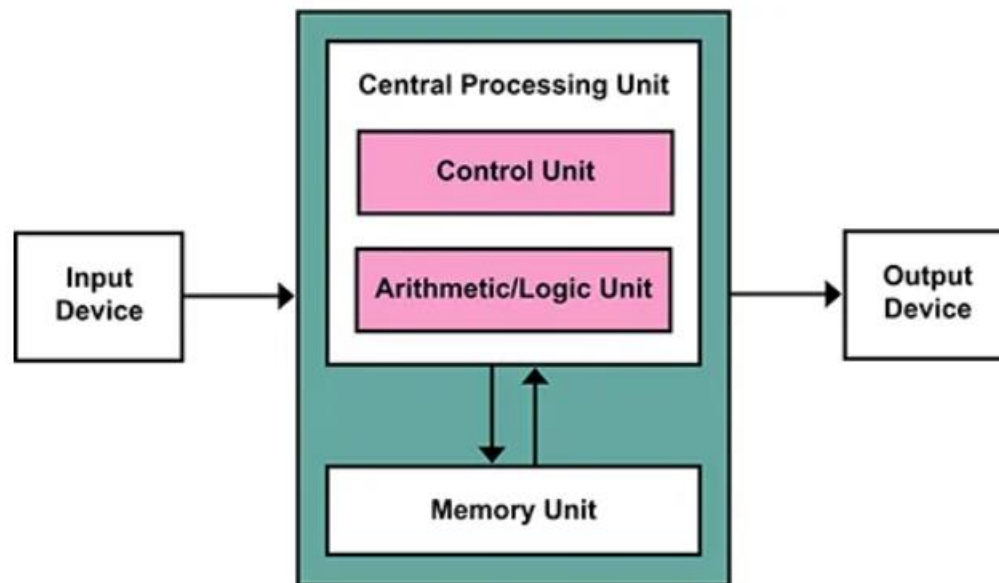
1. Understand what a **program** is, what it consists of and how it relates to the computer
2. Understand the concepts of **input**, **processing** and **output**
3. Understand the concepts of **data** and **data types**
4. Understand what **variables** and **constants** are and how they are declared and used in a program
5. Know how to give **names** to variables and constants
6. Understand how **assignment statements** work
7. Be able to do simple **string manipulation** in a program
8. Be able to do simple **arithmetic** in a program
9. Know how to use the following: ComboBox, RadioButton, GroupBox and the MessageBox **function**

What is a Program?

- A set of instructions to achieve some goal using a computer. To do that it:
 - provides instructions to be executed in a computer's central processing unit (**CPU**)
 - gets data from **input devices**
 - stores data temporarily in **memory** during execution
 - sends data to **output devices**
 - optionally stores data permanently on **storage devices**.



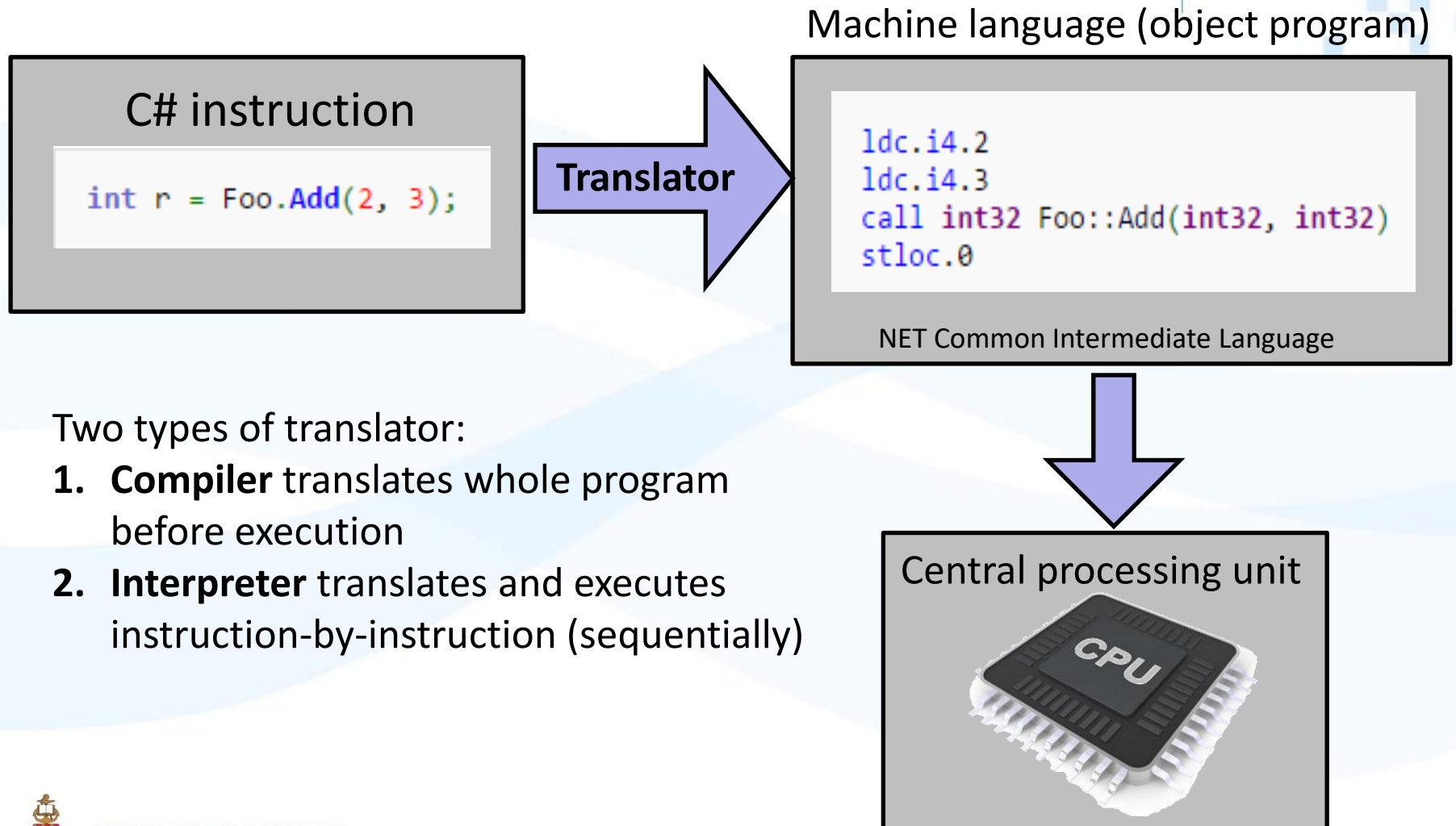
Basic computer architecture



Source [Wikipedia](#)



What happens when a program runs?



Two types of translator:

1. **Compiler** translates whole program before execution
2. **Interpreter** translates and executes instruction-by-instruction (sequentially)



Programming language morphology

The **kind of words and special characters** that a programming language can contain (syntax).

It is **similar to the words and punctuation** in natural language. The main components are:

- **Keywords** (fixed set of words in the language: **C# reserved keyword**)
- **Variables** (named **memory locations**: **int x = 19;**)
- **Constants** (literal data values: **double pi = 3.14;**)
- **Operators** like = + - / * % != == || &&
- Other **special characters** like () { } ; []

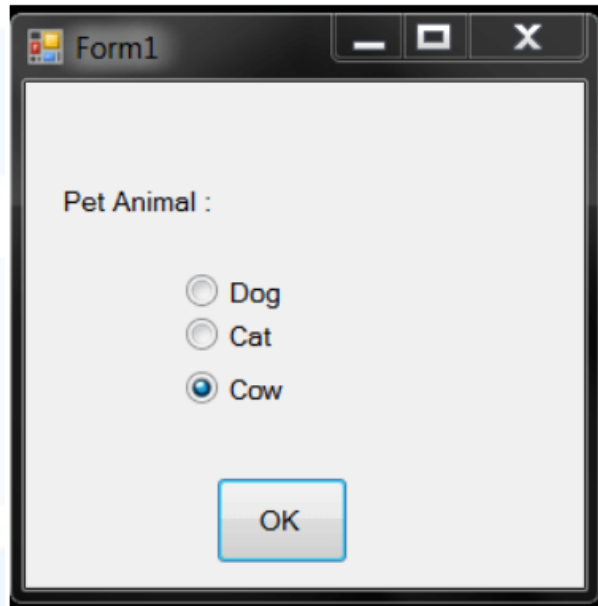
C# synthax

```
/*  
 * C# Program to Create Radio Button and Demonstrate its Use  
 */  
  
using System;  
using System.Collections.Generic;  
using System.ComponentModel;  
using System.Data;  
using System.Drawing;  
using System.Linq;  
using System.Text;  
using System.Windows.Forms;  
namespace WindowsFormsApplication5  
{  
    public partial class Form1 : Form  
    {  
        public Form1()  
        {  
            InitializeComponent();  
        }  
  
        private void label1_Click(object sender, EventArgs e)  
        {  
        }  
  
        private void button1_Click(object sender, EventArgs e)  
        {  
            MessageBox.Show("You Have Clicked Your Pet Animal Using Radio Button ");  
        }  
    }  
}
```



Test Case

Runtime Test Cases



Form1

Pet Animal :

☐ Dog

☐ Cat

☒ Cow

OK

You Have Clicked Your Pet Animal Using Radio Button

OK



Programming language syntax

- The **structure of a programming language (attention to details)**. The valid ways in which the **morphological units** (keywords) can be put together. It is **similar to the sentences and paragraphs** of natural language.
- Programs consist of a **hierarchy** of:
 - **Statements** (`string myname = "Arnold";`)
 - **Methods** (Named blocks of statements. Event handlers are examples of these.) → `MessageBox.Show(myname);`
 - **Classes/objects** (Programming units made up of data and methods. For example, **the controls in the Toolbox represent classes and when we add a specific control to a form we create an object of that class.**) → The object is an instance of a class.
 - **Namespaces** (groups of classes)



C# Windows Forms program code

A C# Forms program typically consists of the following parts:

- References to standard **namespaces**
- This program's **namespace**
- Classes such as the Form **class**
- **Methods**, such as
 - The current form's constructor method
 - Event handlers associated with controls on the current Form

NOTE the curly brackets and how they demarcate the code segments (attention to details).

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace Prac1Exercise1
{
    3 references
    public partial class frmColourFont : Form
    {
        1 reference
        public frmColourFont()
        {
            InitializeComponent();
        }

        1 reference
        private void btnFont_Click(object sender, EventArgs e)
        {
            fontDialog1.ShowDialog();
            txtName.Font = fontDialog1.Font;
        }
    }
}
```



Program statement

- Programs consist of **program statements**, each executing one action in the program.
- A **statement**
 - always **ends with a semicolon**;
 - can go over **many program lines (Inheritance, Polymorphism, and Encapsulation)**

(More than one statement can be on one line of the program, but that is not advisable.)

- Examples (how to declare a variable):

`int sum, input1, input2;` -> All 3 variables are int

`lblSurname.Text = txtInput.Text;` -> Assigning the input to the Form label (input text not int)

`sum = input1 + input2;` -> Sum of two variables stored in the sum variable. (int variables) (+, -, /, %, etc...)

Statement type

Program statements in a program can be one of the following statement types (which will be covered as we go along):

Sequence

Declarations (double y= 75.6), Assignments (=), Arithmetic and mathematical operations (==, !=, &&, ||), String and character manipulations (upper case, reverse, etc), Method calls

Decision

if, switch -> if x==3 then MessageBox.show("ok")

Repetition

while, for, foreach -> While x==3 do MessageBox.show("ok")

Class and object handling

Comments are special statements that appear in the program as explanation of the code. These statements **are not translated and executed** by the computer but are there for the programmer's sake.

C# comments

```
public Form1()
{
    InitializeComponent();
}

private void Form1_Load(object sender, EventArgs e)
{
    // Creating and setting the properties of label
    Label l = new Label();
    l.Location = new Point(122, 80);
    l.AutoSize = true;
    l.Text = "Select Programming Language";

    // Adding this label to the form
    this.Controls.Add(l);

    // Creating and setting the properties of comboBox
    ComboBox mybox = new ComboBox();
    mybox.Location = new Point(327, 77);
    mybox.Size = new Size(216, 26);
    mybox.Items.Add("C#");
    mybox.Items.Add("Java");
    mybox.Items.Add("Scala");
    mybox.Items.Add("C");
    mybox.Items.Add("C++");

    // Adding this ComboBox to the form
    this.Controls.Add(mybox);
}
```



Output

Select Programming Language

C#
Java
Scala
C
C++



Input – processing - output

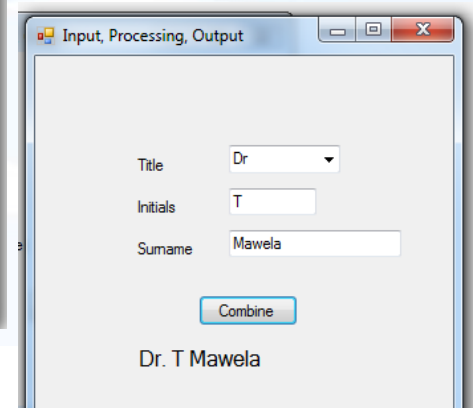
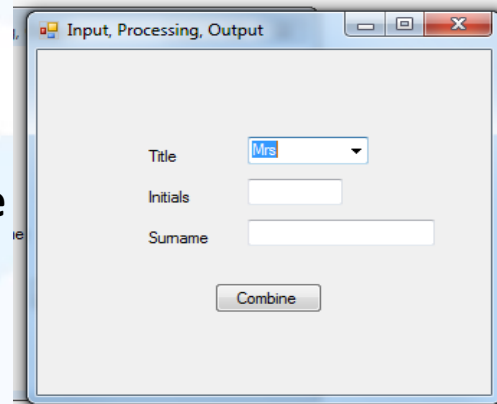
- All programs are concerned with the following:
 - Input**: data comes into the program via **input devices**
 - Processing**: the input data is processed by the **CPU**, often storing data temporarily in the **memory**
 - Output**: the results of the processing are displayed on an **output device** such as a screen or printer
- For example
 - Input**: User's birth date is entered into the program via the keyboard (int = birthdate)
 - Processing**: The program calculates the number of years between user's birth date and today's date
-> int yourage = this year – birthdate (processing);
 - Output**: The user's age is displayed on a screen ("your age is:" +yourage)

Exercise 1: Input – processing - output

Create a Form named frmInputOutput and add the following controls to it:

- A **ComboBox** with Mrs, Ms, Miss, Mr, Dr, Prof in its Items property (this will provide a dropdown box from which the user can select an item).
- Two **TextBoxes** – one for initials and one for a surname
- A Button named **btnCombine** and with **Text set to Combine**
- **Three labels for the three input fields**
- **One Label** for the output string

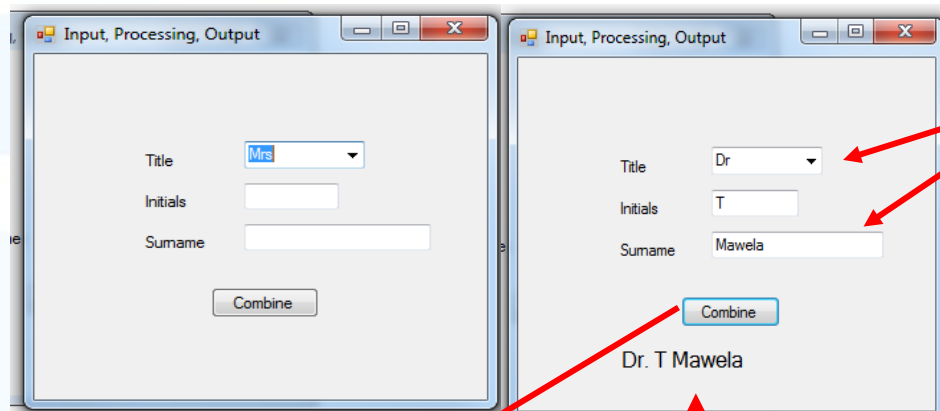
When the **user clicks the button the three input strings are combined** with a fulstop after the title (see alongside).



```
private void btnCombine_Click(object sender, EventArgs e)
{
    string combinedName;
    combinedName = cboTitle.Text + ". " + txtInitials.Text + " " + txtSurname.Text;
    lblOutput.Text = combinedName;
}
```

Input – processing - output

Example program : Take three input values – a title, initials and surname (data type?) – process them by combining into one string and display the result



Input from keyboard and mouse, through ComboBox and TextBoxes

Data in memory

Processing

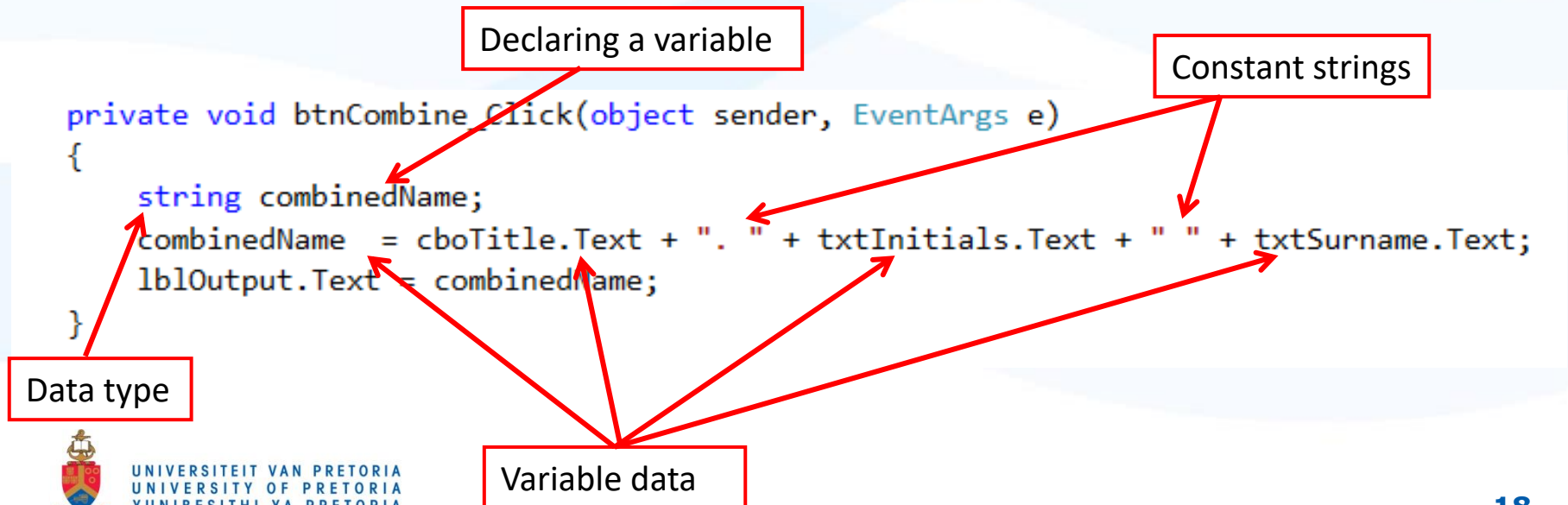
```
private void btnCombine_Click(object sender, EventArgs e)
{
    string combinedName;
    combinedName = cboTitle.Text + ". " + txtInitials.Text + " " + txtSurname.Text;
    lblOutput.Text = combinedName;
}
```

Output to screen through a Label



Data: variables and constants

- All input and output are data and all processing involves data
- Data in programs can either be variable or constant
- **Constant** data cannot change when the program runs
- A **variable** is a named location in computer memory that can hold different values as the program runs (variables can change but constants cannot)
- Constant and variable data has a **data type**, for instance, integer (int x=10), double (double x=10.20), date and string (string yourname)

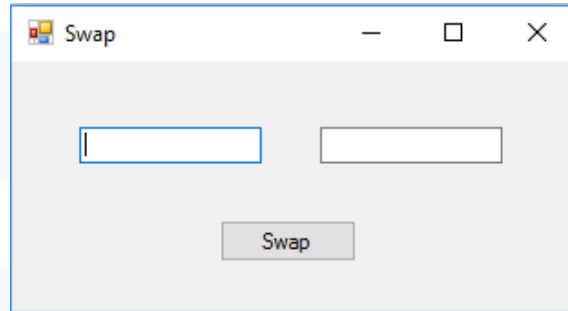


Some data types

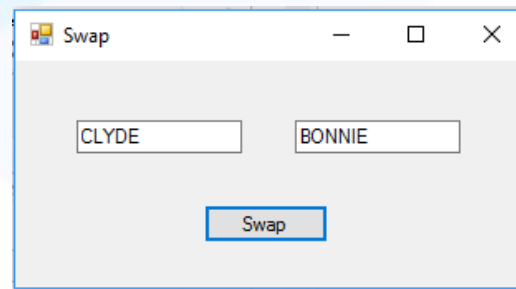
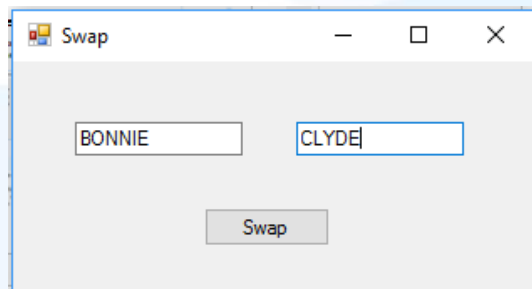
TYPE	DESCRIPTION	POSSIBLE VALUES	EXAMPLES
int	Integer	-2 147 483 647 to 2 147 483 648	30006
double	Decimal	-7.9×10^{28} to 7.9×10^{28}	3425.25
bool	Boolean	True or False	True
string	String of characters	Any string of Unicode characters	“My No. 1 Favourite”
char	One character	Any one character	‘m’

Exercise 2: Variables

Create a simple Forms program with two TextBoxes and a Button on the Form as shown below.



When the user clicks Swap the values entered into the TextBoxes are swapped.



Exercise 2: Variables

Maybe your first try looked like this:

```
private void btnSwap_Click(object sender, EventArgs e)
{
    txtInput1.Text = txtInput2.Text;
    txtInput2.Text = txtInput1.Text;
}
```

But this doesn't work because you **end up with the same value in both TextBoxes**. So, before we assign the second string to the first TextBox we have to **store that value in the first TextBox for later use**. For that we need a **temporary variable**. Then, if we have copied the second string into the first TextBox, we can copy that stored initial value of the first TextBox to the second TextBox.

```
private void btnSwap_Click(object sender, EventArgs e)
{
    string placeholder;
    placeholder = txtInput1.Text;
    txtInput1.Text = txtInput2.Text;
    txtInput2.Text = placeholder;
}
```

Variables

- Data that are processed in a computer program needs to be kept in the computer's **random access memory (RAM)** for the duration of the program execution
- Each data element is assigned a section of RAM and in order to use that data in our **program it has to be named**
- Data elements that can change during execution of a program are **variables**
- So, when you declare a **variable, you allocate memory** for that data element and you give it a name
- The following declaration statement allocates a section of memory that is big enough to keep a string value, to the variable myName.

string myName;

- Now you can use the variable in the program, e.g.

myName = txtInput.Text;

- Because the variable was declared as a string you can only assign string values to it

Constants

Constants are data values that do not change during program execution. You can use constant values directly in the code or you can give them a name.

For example:

```
const double TAX_RATE = 0.40;  
const string QUESTION = "What is the sales amount?";  
const char HIGH_SYMBOL = 'A';  
double taxAmount;
```

They can be used as follows:

```
taxAmount = income * TAX_RATE;  
txtDialog.Text = QUESTION;  
lblResult.Text = "The result is " + HIGH_SYMBOL;
```

Declaration

- Examples of declarations

```
string myAnswer;  
double result;  
const string MESSAGE = " Well done! ";  
bool memberOfTeam;  
const double TAX_RATE = 0.14;  
char prefix;
```

- Naming conventions

Constants: all uppercase and use _ to separate words. For example:
TAX_RATE, SPEED_LIMIT, QUESTION1

Variables: **start with lower case letter, every new word starts with uppercase letter**. For example:

value1, courseName, averageScore, count

-> 1 value? 30course name?

Note: Cannot use any of the **C# reserved keywords** as names

The scope of variables and constants

- The variables that we have used so far were all declared inside an event handler. We call these variables **local variables**.
- Local variables only exist while the event handler (or method) in which they are declared is being executed.
- A local variable declared in one event handler (or method) can therefore **not be directly accessed by any other event handler** (or method).
- It is possible to declare a private variable on Form level (i.e. in the Form class's code) **where all the event handlers in the Form will have access to that variable. (Global variable)**
- Getting the variable scope right and understanding the problems that can arise when using globally declared variables carelessly, is important in programming – all of this will unfold as we progress through the course.

The Assignment statement

- The assignment operator assigns values to variables or control properties
- The assignment operator is the equal sign “=”
- Any value to the right of the operator is assigned to the variable or control property on the left

```
myAnswer = txtAnswer.Text;  
result = 2365 * TAX_RATE;  
lblResult.Text = "The end";  
memberOfTeam = true;  
prefix = '2';
```

IMPORTANT: The variable/property on the left and the variable/value on the right must be of the same data type (int, double, string, Boolean).

Arithmetic

OPERATOR	DESCRIPTION	EXAMPLE	VALUE
+	Addition	3 + 5 6.5 + 7.3	8 13.8
-	Subtraction (or negation)	5 - 3 -35	2 -35
*	Multiplication	3 * 5	15
/	Division	15/3	5
%	Modulus (remainder)	15%3 16%3	0 1

Note:

1. Arithmetic can only be done on integer and decimal (double) data
2. + is an overloaded operator, meaning it can also be used to combine two strings together (see next slide)



Rules

For Example; $15 \div 3 \times 1 \div 5$

'**Multiplication**' and '**Division**' perform equally, so calculate from left to right side. First solve $15 \div 3 = 5$, then $5 \times 1 = 5$, then $5 \div 5 = 1$.

For Example; $7 + 19 - 11 + 13$

'**Addition**' and '**Subtraction**' perform equally, so calculate from left to right side. First solve $7 + 19 = 26$, then $26 - 11 = 15$ and then $15 + 13 = 28$.

For Example; $(6 + 4) \times 5$

First solve inside 'brackets' $6 + 4 = 10$, then $10 \times 5 = 50$.

Next solve the mathematical 'Of'.

For Example; 3 of $4 + 9$

First solve 'of' $3 \times 4 = 12$, then $12 + 9 = 21$.



String concatenation

- Often you want to add different strings together to form one longer string
- This is called **concatenation**
- The operator for concatenation is +
- Note: The + operator has two meanings
 - For numeric data + means **addition**
 - For string data + means **concatenation**

12 + 34 will give the result → 46

“12” + “34” will give the result → “1234”

“ban” + “ana” will give the result → “banana”



Conversion between types

- Sometimes the user will provide **numeric input by typing the number into a TextBox.**
- The program will **access the number through the TextBox's Text property which is of type string.**
- Before that number can be used in an arithmetic statement, **it has to be converted to a double or integer value first.**
- We use **the Convert.ToInt32()** method to convert a string to an integer.
- We use the **Convert.ToDouble()** method to convert a string to a double.
- We use the **Convert.ToString()** method to convert an integer or a double value to a string.
- See the example on the next slide.

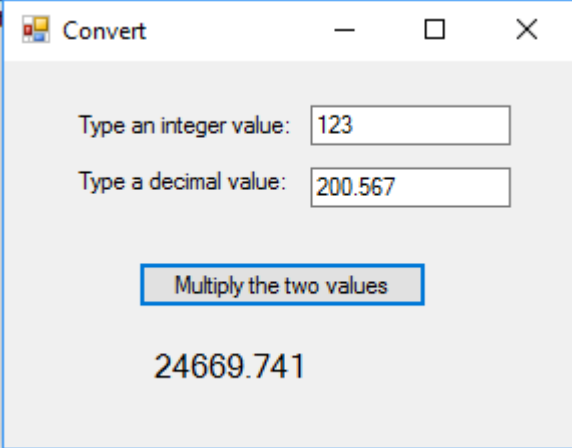
Conversion between types

We use a simple example to illustrate:

In the form the user provides an integer and a decimal value.

By clicking the button these values are multiplied with each other and the answer displayed in a Label.

The event handler shows all the conversions required.



```
private void btnMultiply_Click(object sender, EventArgs e)
{
    int value1;
    double value2;
    double answer;

    value1 = Convert.ToInt32(txtValue1.Text);
    value2 = Convert.ToDouble(txtValue2.Text);
    answer = value1 * value2;
    lblResult.Text = Convert.ToString(answer);
}
```



Input and output mechanisms

So far you have used TextBoxes, Dialogs and a ComboBox to provide input to the programs we have written.

There are many other controls that can be used for input, such as:

RichTextBox (for text input that involves edited text or paragraphs of text)

RadioButtons (when the user has to choose one of several options)

CheckBoxes, GroupBox

OpenFileDialog (to open an external file)

And more mechanisms for output are, for example:

MessageBox (note that this is not a control in the Toolbox)

SaveFileDialog (to save a file to and external disk)

You will get to use these and many more in the course of the module.

Comments

- Comments are notes that are included in a program to explain statements or groups of statements in a program
- Comments are not part of the program code
- Inline comments appear on a single line and start with //
- For example:

// Declaration section

- Multiline comments go over two or more lines. They start with /* and end with */. For example:

/* Assignment 1 for INF154

Submitted on 16 Feb 2017

Student number: 41839222 */

- All the programs that you write from now onwards have to contain meaningful comments.

Errors

- It is quite easy to accidentally type something incorrectly when entering code into your computer. If this happens, the compiler will report a ***syntax error*** when it tries to compile it (**logical error?**)
- Errors are reported in the Error list at the bottom of the screen
- The message displayed will contain the line number and character position at which the error is found and a description of the error.
- If you double-click on the error it will take you to the line in error

The error here is that we are trying to assign the txtInput1 control to a string variable instead of its Text property – **txtInput1.Text**

```
23 placeholder = txtInput1;
24 txtInput1.Text = txtInput2.Text;
25 txtInput2.Text = placeholder;
26 }
27 }
28 }
29 }
```

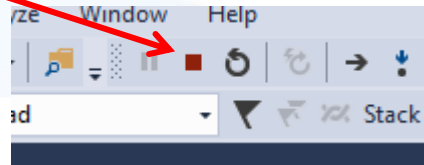
Error List

1 Error 0 Warnings 0 Messages

Code	Description	Project	File	Line	Suppression St...
CS0029	Cannot implicitly convert type 'System.Windows.Forms.TextBox' to 'string'	Lecture2Exercise2	Form1.cs	23	Active

Errors

- When a run time error occurs you can stop the program by clicking on the **“Stop Debugging”** button or press Shift-F5.

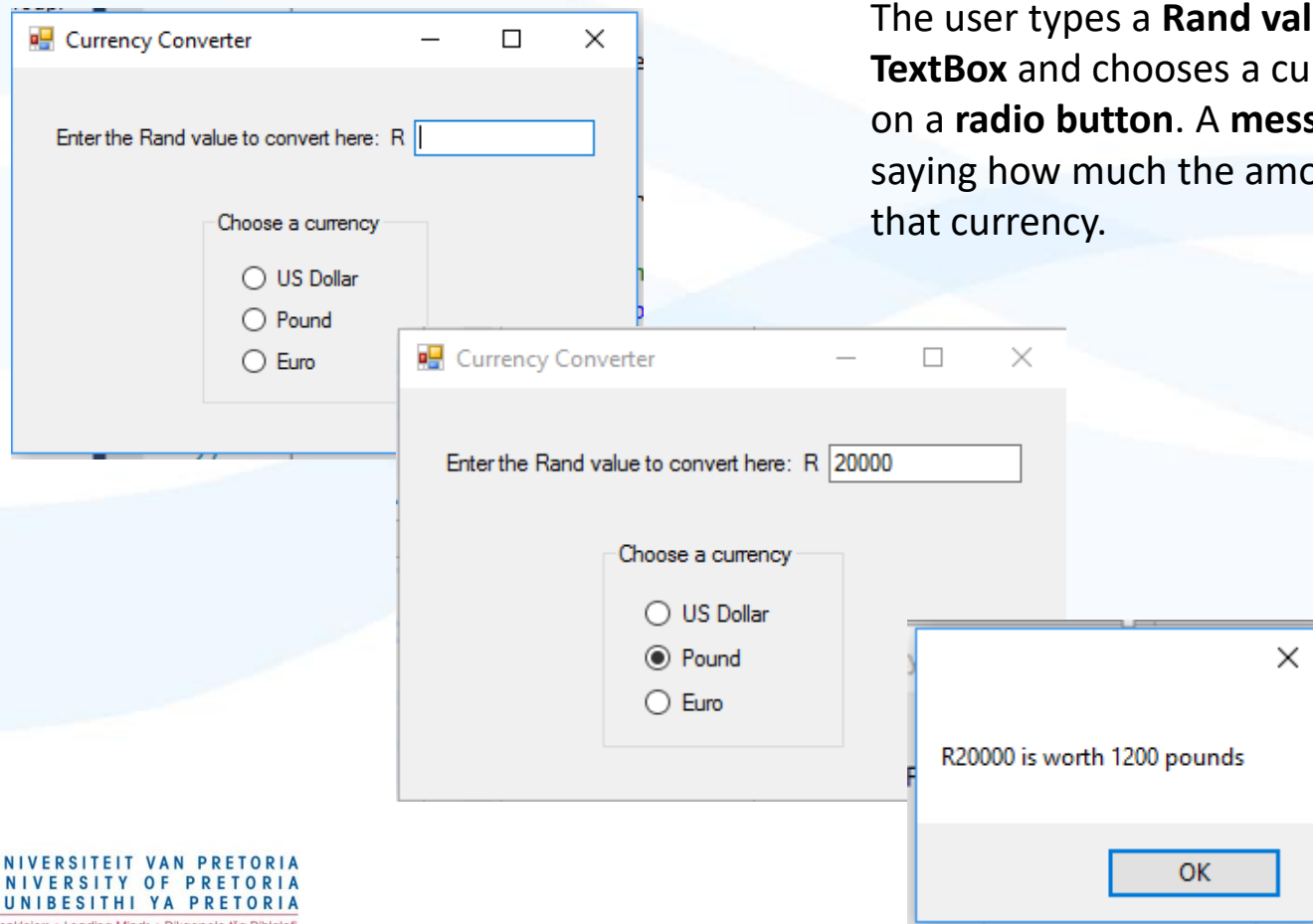


- In the course of INF 154 and INF 164 you will learn **how to debug your code through inspection**, how to include code that will catch errors before they occur (through exception handling) and how to use the built-in Visual Studio debugger.

Lecture 2 - Exercise 3 : Try it at home

To practise using variables and learn to use RadioButtons and the MessageBox function we are now going to write a currency converter program that works as follows:

The user types a **Rand value** into the **TextBox** and chooses a currency by clicking on a **radio button**. A **message box** pops up saying how much the amount is worth in that currency.



Exercise 3 – RadioButtons, GroupBox, etc.

Create a new project with the controls listed below:

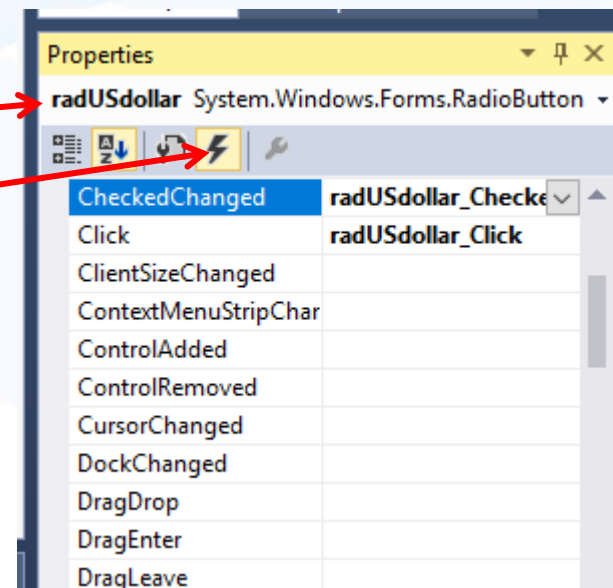
Control type	Name property	Other properties
Form	frmCurrency	Text: “Currency Converter”
Label	lblPrompt	Text: “Enter the Rand value to convert here: R”
TextBox	txtRandValue	TabIndex: 0 (so that the cursor appears in it initially)
GroupBox	gbxCurrency	Text: “Choose a currency” TabIndex: 1
RadioButton	radUSDollar	Text: “US Dollar”
RadioButton	radPound	Text: “Pound”
RadioButton	radEuro	Text: “Euro”



Note that the **RadioButtons** must be placed inside the **GroupBox**.
When you then move the **GroupBox** using the handle at the top left, all the **RadioButtons** will move with it.

Exercise 3 – RadioButtons, GroupBox, etc.

- When the user clicks on one of the RadioButtons, something must happen, so we are going to write the code for this program in the three Click event handlers of the RadioButtons.
- When we created a Click event handler for the Button control we just had to double-click on the Button.
- The Click event handler for a RadioButton is created in a different way, because when you double-click on the RadioButton it will open the CheckedChanged event handler and not the Click event handler.
- To create a Click event handler for the first RadioButton do the following:
- Select that RadioButton so that its name appear at the top of the Properties window
- Click on the thunderbolt icon, so that instead of the properties you see a list of event names. The second one is Click.
- Double-click on it to open the event handler.



Exercise 3 – RadioButtons, GroupBox, etc.

Add the following code to the event handler, Save All, run the program to test the first radio button.

```
private void radUSDollar_Click(object sender, EventArgs e)
{
    // Constant and Variable declarations
    const double US_EXCH_RATE = 0.074; // As on 18/1/2017
    double randValue, dollarValue;
    string outputString;

    // Get input, convert to double, calculate $ value
    randValue = Convert.ToDouble(txtRandValue.Text);
    dollarValue = randValue * US_EXCH_RATE;

    //Build output string and display in message box
    outputString = "R" + txtRandValue.Text + " is worth " + "US$" +
        Convert.ToString(dollarValue);
    MessageBox.Show(outputString);
}
```



Exercise 3 – RadioButtons, GroupBox, CheckBox

Handling errors

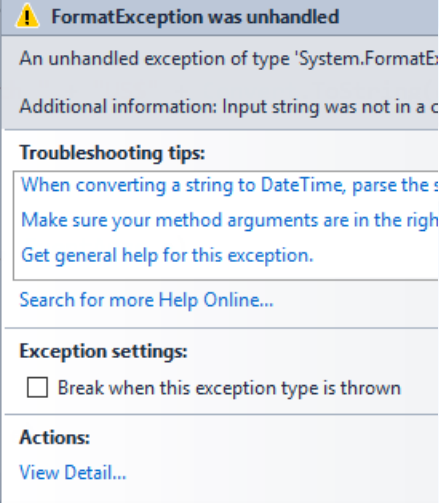
```
private void radUSDollar_Click(object sender, EventArgs e)
{
    // Constant and Variable declarations
    const double US_EXCH_RATE = 0.074; // As on 18/1/2017
    double randValue, dollarValue;
    string outputString;

    // Get input, convert to double, calculate $ value
    randValue = Convert.ToDouble(txtRandValue.Text);
    dollarValue = randValue * US_EXCH_RATE;

    //Build output string and display in message box
    outputString = "R" + txtRandValue.Text + " is worth " + dollarValue + " dollars";
    MessageBox.Show(outputString);
}
```

```
1 reference
private void radUSDollar_CheckedChanged(object sender, EventArgs e)
{
}
```

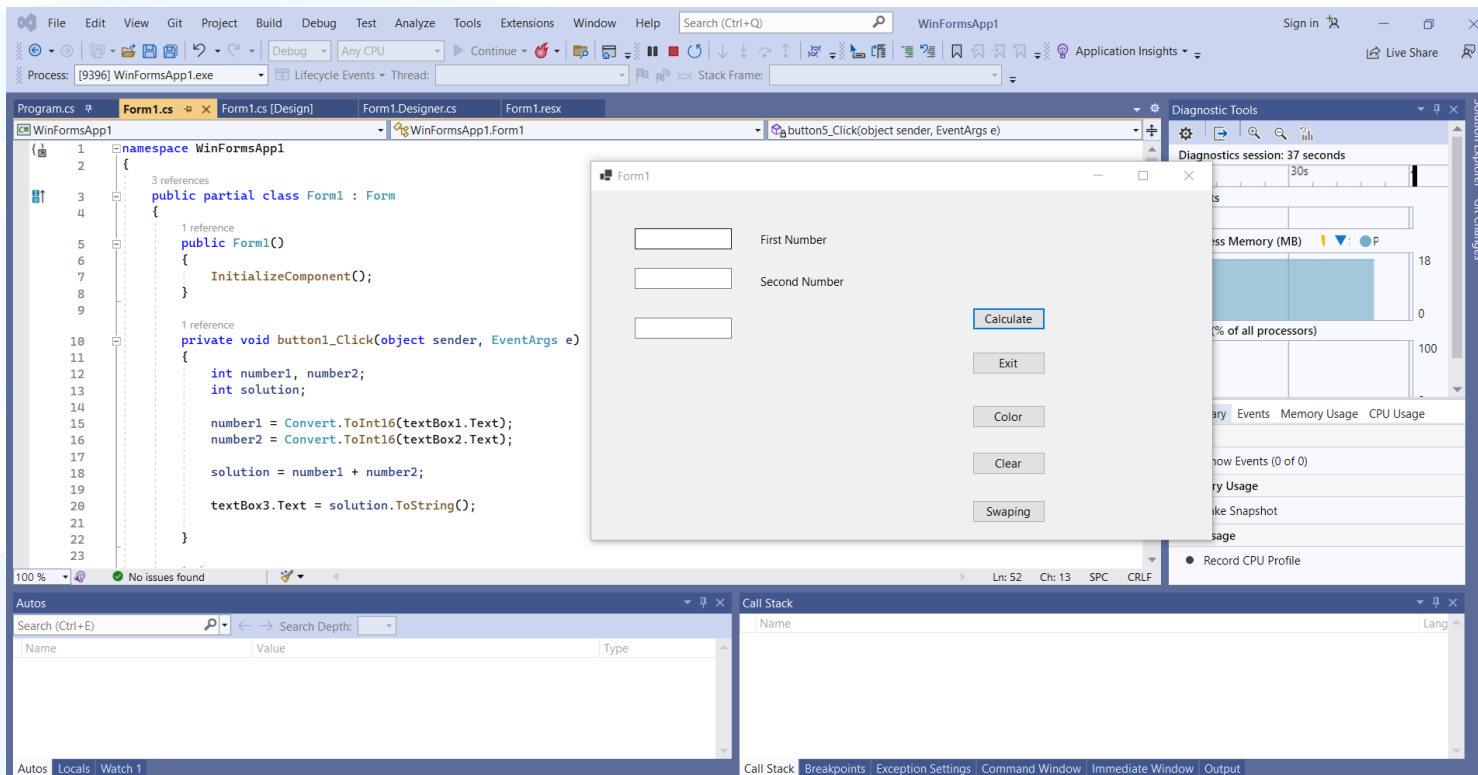
```
1 reference
private void radGroup2_Click(object sender, EventArgs e)
{
}
```



This run-time error will occur if the user clicks on a radio button before entering a value in the text box. The program statement attempted to convert an empty string into a double value.

For now, the easiest way to prevent this is to put a 0 in the Text property of the txtRandValue TextBox as a default input value.

Try it



Try it

